# COMP364 Quiz 4

**Question 1. Computing convolution output values**

Each of the following questions describes two adjacent layers in a one-dimensional convolutional neural network: upstream layer $A$ and downstream layer $B$. In each case, give the values computed at $B$ based on inputs from $A$. Figure 1 summarizes the relevant channel values and kernels used in each part of the question.

$$
\begin{aligned}
A_0 &= (5, 0, -1, 1, 0) \\
k_{00} &= (1, -2, 1) \\
b_0 &= 2 \\
s &= 1
\end{aligned}
$$

$$
\begin{aligned}
A_0 &= (5, 0, -1, 1, 0) \\
A_1 &= (0, -2, 4, 0, 1) \\
A_2 &= (6, 4, 3, -3, 0) \\
k_{00} &= (1, -2, 1) \\
k_{10} &= (4, 0, -3) \\
k_{20} &= (0, 1, 2) \\
b_0 &= 2 \\
k_{01} &= (3, 3, 3) \\
k_{11} &= (-1, 0, -1) \\
k_{21} &= (2, 1, 0) \\
b_1 &= 0 \\
s &= 2
\end{aligned}
$$

Figure 1: **Top:** Channel value and kernel for question 1(a). **Bottom:** Channel values and kernels for questions 1(b)–(e).

(a) **(5 points)** Layer $A$ has one channel, $A_0$, of length 5. Its output is $A_0 = (5, 0, -1, 1, 0)$. $B$ has 1 channel, $B_0$. Channel $B_0$ is produced from channel $A_0$ by cross-correlating with a kernel of size 3; the kernel is $k_{00} = (1, -2, 1)$. Channel $B_0$ has bias $b_0 = 2$. Layer $B$ employs a stride of

| | channels in $A$ | length of $A$ | kernel width | kernel stride | padding (at each end) | bias | channels in $B$ | length of $B$ | num parameters in $B$ | num ignored inputs |
|---|---|---|---|---|---|---|---|---|---|---|
| (a) | 1 | 64 | 8 | 1 | 0 | False | 1 | | | |
| (b) | | | | | | True | | | | |
| (c) | 6 | | | | | | | | | |
| (d) | | | | | | | 10 | | | |
| (e) | | | | | | False | | | | |
| (f) | | | | 2 | | | | | | |
| (g) | | | 7 | 1 | | | | | | |
| (h) | | | | 2 | | | | | | |
| (i) | | | | | 1 | | | | | |
| (j) | | | | | 2 | | | | | |
| (k) | | | | | 3 | | | | | |

Figure 2: Details for question 2. Fill in the last 3 columns.

$s = 1$. More generally, the notation $k_{ij}$ denotes the kernel mapping channel $A_i$ to channel $B_j$. The symbol $b_i$ designates the bias of channel $B_i$, and $s$ designates the stride for layer $B$.[1]

(b) **(5 points)** Everything is identical to the previous question except that the kernel stride is $s = 2$.

(c) **(5 points)** Everything is identical to (b) except that $A$ now has a second channel with outputs $A_1 = (0, -2, 4, 0, 1)$, and $B$ has a corresponding new kernel $k_{10} = (4, 0, -3)$.

(d) **(5 points)** The same as (c) except now we add a third channel to $A$: this is channel $A_2$ with outputs $A_2 = (6, 4, 3, -3, 0)$ and kernel $k_{20} = (0, 1, 2)$.

(e) **(10 points)** The same as (d) except now we add a second channel to $B$, so $A$ has three channels with IDs 0, 1, 2; and $B$ has two channels with IDs 0, 1. The bias for channel $B_1$ is $b_1 = 0$, and the new kernels for channel $B_1$ are

$$k_{01} = (3, 3, 3)$$
$$k_{11} = (-1, 0, -1)$$
$$k_{21} = (2, 1, 0)$$

**Question 2. Computing convolution output sizes (33 points)**

Figure 2 on page 2 describes various configurations for two consecutive layers, $A$ and $B$, in a one-dimensional convnet. Each row of the table shows only the *changes* in configuration values compared to the previous row. So you can assume that all other configuration values remain the same as the previous row. For example, configuration (b) is the same as (a) except that (b) employs

[1] In the engineering literature, the word *convolution* refers to an operation in which the kernel is flipped. The correct technical term for applying a kernel without flipping is *cross-correlation*, not convolution. However, it is common in the CNN literature to use cross-correlation and convolution interchangeably, and the kernels are never flipped. We use this standard CNN convention here, so kernels are not flipped.

```
import torch
import torch.nn as nn
loss_fn = nn.MSELoss()
target = torch.tensor([[1, 2], [20, 21]], dtype=torch.float32)
input = torch.tensor([[1.1, 2], [20, 20]], dtype=torch.float32, requires_grad=True)
num_vals = input.numel()
print(f"num_vals: {num_vals}")
loss = loss_fn(input, target)
loss_val = loss.item()
print(f"loss_val*num_vals: {loss_val * num_vals}")
print(f"loss_val: {loss_val}")
loss.backward()
print(f"dLoss/dInput * num_vals: \n{input.grad * num_vals}")
print(f"dLoss/dInput: \n{input.grad}")
optimizer = torch.optim.SGD([input], lr=0.1)
optimizer.step()
print(f"Updated input: \n{input.detach()}")
```

Figure 3: PyTorch code for question 4

biases in its units. The changes are cumulative. For example, (c) is the same as (b) except that (c) has 6 channels, and (c) *does* employ biases, as this property is inherited from (b).

The task for this question is to fill in the last three columns of figure 2 on page 2.

### Question 3. Computing convolution output size with additional layer (10 points)

Consider a convolutional network that has layers $A$ and $B$ exactly as in the final part of the previous question, 2(k). We add another layer $C$, which has 5 channels and employs a kernel of width 7, stride 2, no bias, and padding 3 on each side. Give the (a) shape and (b) parameter count of layer $C$.

### Question 4. Regression using MSE and PyTorch

(a) **(2 points)** Given output values $x_1, x_2, \ldots, x_n$ and target values $t_1, t_2, \ldots, t_n$, what function is minimized when using mean squared error (MSE)?

(b) **(15 points)** Figure 3 shows some code for performing regression in PyTorch using mean squared error (MSE), and figure 4 shows the output of this code. Give a brief explanation of each line of output.

Total number of points: **90**

```
num_vals: 4
loss_val*num_vals: 1.0099999904632568
loss_val: 0.2524999976158142
dLoss/dInput * num_vals:
tensor([[ 0.2000,  0.0000],
        [ 0.0000, -2.0000]])
dLoss/dInput:
tensor([[ 0.0500,  0.0000],
        [ 0.0000, -0.5000]])
Updated input:
tensor([[ 1.0950,  2.0000],
        [20.0000, 20.0500]])
```

Figure 4: Output of code in figure 3